

However, if we defined the same variable outside that function, it would be visible from *within* the function.

```
const a = 1;

function scopeTest() {
  console.log(a);
}

scopeTest();
-> 1
```

In the first case, the variable is block-scoped, that is, confined to the block of the function. In the latter case, the variable is defined globally (outside all functions) and can be accessed anywhere in the program.

Although it may seem more beneficial to be able to access a variable from anywhere within a program, we should exercise caution where global variables are concerned. In general, it is better to explicitly restrict the scope of our variables. This is one of the main reasons why `var` has been deprecated, as it can't be "contained" in the same way that `let` and `const` can be.

```
function scopeTest() {
  var a = 1;
}

console.log(a);
-> 1
```

As you can see earlier, when using `var`, the variable can break out of the surrounding block, and this can be problematic in larger programs.

When coding generative art, we're usually writing relatively small programs (or "sketches") and global variables aren't as much of an issue. However, we'll discuss in the next section how best to make certain settings globally available throughout our sketches without relying on global variables.

Anonymous Functions

Functions don't always have to have a name. Particularly when working with events and callbacks (which we'll cover in a later chapter), functions are often called anonymously as function expressions, executed in order to return a value. Anonymous functions can also be assigned to variables that then store the returned value.

```
// An anonymous function expression assigned to a variable.  
const sum = function(a, b) {  
  return a + b;  
}
```

In the preceding example, the variable can be used to execute the function.

```
sum(5, 4);  
-> 9
```

One thing we cannot do with anonymous functions is use them at the start of a statement.

```
// The below is a no-no.  
function(a, b) {  
  return a + b;  
}  
-> Uncaught SyntaxError: Function statements require a  
function name
```

Arrow Functions

So far we've covered functions declared with the `function` keyword, but there is another, more modern way of writing functions. These functions are called arrow functions due to the arrow syntax they employ.

```
// Basic syntax of an arrow function.  
const performSomeTask = () => {  
  // ... code defining the task.  
}
```

They are concise alternatives to the traditional function expressions we've just covered. Let's rewrite the `sum()` function as an example.

```
const sum = (a, b) => {  
  return a + b;  
}
```

A little shorter, as you can see. But it can get even more concise than this if our function only consists of a single statement that returns a value (which exactly describes our `sum()` function). In such a case, we don't need the curly braces or even the `return` statement itself.

```
const sum = (a, b) => a + b;
```

Because the arrow function body consists of just a single statement (`a + b`), it is assumed we want the value of this statement returned. This is what's called an implicit return.

Now, not all arrow functions can be so neatly condensed, just those that have a single return statement. But such one-liners are liberally used throughout most modern JavaScript code bases, particularly those that use declarative front-end frameworks like React, so it's worth familiarizing yourself with them.

There is another reason to use arrow functions that stretches a little beyond the scope of this chapter, and it's this. What I mean is the `this` keyword, which is a complicated topic in JavaScript (and complicated to explain without getting tangled in reflexive language). Suffice to say, the value of `this` is more predictable and less prone to bugs when used in arrow functions. But don't worry about this for now.

Instead, let's concern ourselves with some data structures that will serve us well in generative art and in JavaScript more generally. This is the subject of our next section.

Objects and Arrays

Variables are great when you need to define values like numbers, strings, and booleans, but what if you need a more sophisticated way to represent and organize your data? Say you have a long list of things – be they this week's groceries, the employees in your company, the locations you want to visit before you kick the proverbial bucket, or, to bring it back to generative art, the hexadecimal codes of the 216 web-safe colors you'd like to sample for your next retro-inspired masterpiece (in the 1990s, web developers – or web masters as they were then known – had a much more restricted color palette).

When you have lists of anything, arrays are your friend.

Arrays

An array is essentially a variable that can contain multiple values. And in JavaScript, these values needn't be of a uniform type. This means we can mix numbers, strings, and booleans into a single array if required (though it's not necessarily recommended you do this).

Arrays take the following format:

```
// The syntax of a standard array.  
let arrayName = [item1, item2, item3, ... itemN];
```

Each array item has a corresponding index, or number, that represents its position in the array. The first item in an array has an index of 0 (counting from zero in computing is the norm), the second item has an

index of 1, the third of 2, etc. To directly access an item in array by its index, you use square bracket notation like so:

```
// Access the first item.
let firstItem = arrayName[0];

// Access the second item.
let secondItem = arrayName[1];

// Access the nth item.
let nthItem = arrayName[n + 1];
```

This lends itself particularly well to loops. Iterating over items in an array with a for loop is a very common practice, and it's something we'll lean on heavily throughout our generative art journey.

Let's continue with our web-safe colors example and iterate over an array containing these 216 hexadecimal values.

You don't need to worry about how we get these items into the array for now; all we're interested in is how we'd loop through them and take some action for each item in the array. The action in this case will be to set the color (i.e., fill) of a newly created circle on each iteration. And to make sure we do this 216 times, we use the length property of the array to control how many times we run our loop.

```
for (let i = 0; i < webSafeColours.length; i += 1) {
  // ... some code.
  circle.fill(webSafeColours[i]);
  // ... more code.
}
```

In Figure 2-1, you'll see the output of this code snippet. Obviously there's more involved in achieving this spiral arrangement (something we'll cover in a later chapter); the main point is how we use the iterator variable `i` to cycle through each color value, that is, `webSafeColours[i]`.

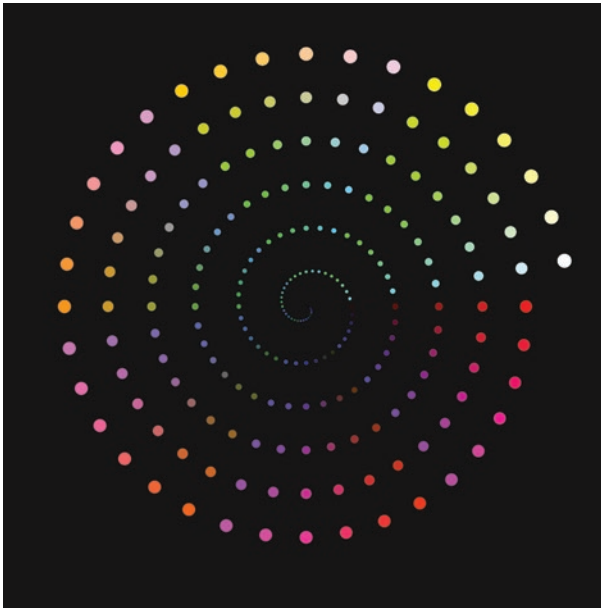


Figure 2-1. *A web-safe color spiral*

If you're brave enough to inspect the code behind the example, you'll notice that to add each color to the array initially, I used an array method called `push()`. The relevant line is

```
colours.push(colour);
```

The `push()` method is how we add an item to the end of an array (think of loading a truck from the back). If instead we wanted to add the item to the beginning of the array, we'd use a method called `unshift()`.

```
colours.unshift(colour);
```

If we wanted to remove an item from the end of the array, we'd use the `pop()` method (think popcorn popping out of a pot). Conversely, if we wanted to remove an item from the beginning of the array, we'd use the `shift()` method (so-called because we're shifting all remaining items to a lower index).

There's one other method I want to cover before moving on, namely, the `forEach()` loop I mentioned earlier. This method is unique to arrays and acts on each item of the array in turn, using a function as its argument. As an aside, functions that are passed as arguments in this manner are known as callback functions.

```
// Set up an sample array.
const someNumbers = [2, 4, 6, 8, 10];

// Call the forEach method on each array item.
someNumbers.forEach((item) => {
  console.log(item * 2);
});

-> 4 8 12 16 20
```

The `forEach()` loop can be a nice succinct way of iterating over an array if the main purpose of the loop is to perform some action specifically for each array item.

Objects

Objects are another data structure JavaScript offers, and like functions, they are a foundational feature of the language. Take a peek under the hood and you'll find objects everywhere. Functions are, in fact, special kinds of objects.

Like arrays, objects can contain multiple values. The primary difference is that objects label their values with names, whereas arrays label their values with numbers (i.e., the index of each item).

An object takes the following format:

```
const objectName = {
  name1: value1,
  name2: value2,
  name3: value3
};
```

These name-value pairs are the object's **properties**. The values can be anything you want: strings, booleans, functions, arrays, or even other objects. Almost anything you can think of can be represented as an object. Let's take my laptop as an example.

```
const myLaptop = {
  processor: "Core i5",
  screenSize: "14 inches",
  inSleepMode: false,
  powerUp: function() {
    if (inSleepMode) {
      wakeUp();
    } else {
      runBIOS();
    }
  }
}
// etc
}
```

With these properties in place, we can call them with either bracket notation, or – more commonly – dot notation.

```
// Accessing an object property with dot notation.
```

```
myComputer.screenSize;
-> "14 inches"
```

```
// Accessing via bracket notation.
```

```
myComputer[processor];
-> "Core i5"
```

We can also update and create new object properties via the same syntax.

```
// Updating an object's property.
myComputer.inSleepMode = true;
```



```
// Creating a new object property.
myComputer.storage = "512 GB SSD";
```

If you take a look back at our first sketch again, you'll notice that I used an object to store some randomized settings that are later called in the sketch:

```
...
const randomised = {
  hue: random(0, 360),
  rotation: random(-180, 180),
  iterations: random(10, 100)
}
...
let rotation = randomised.rotation + (i * 2);
```

This is a good way of getting around the issue of global variables I mentioned earlier. Using an object, we can make our settings globally available without relying on global variables. This can keep our code tidier and less likely to cause conflicts.

Classes

A class is like a blueprint, or template, for creating objects. The objects that are created are then said to be *instances* of that class. A class can include both data and functions to work on that data. This coupling of data and code is referred to as encapsulation.

We won't be going into much detail with classes, but it will be useful to know some basic terminology, as the SvJs library we'll be using is class based.

To create an instance of a class, we use the `new` keyword, followed by the class name with brackets. This calls the class constructor.

```
// Defining a class.  
class StarShip {  
    ...  
}  
  
// Instantiating (or constructing) a class.  
const prometheus = new StarShip();
```

Functions that are part of a class are known as methods. A class will usually have, at the very least, methods to get and set its properties.

```
prometheus.setDestination = 'Moon LV-223, Zeta 2 Reticuli';  
prometheus.takeOff();
```

When we created our first generative SVG in Chapter 1, we began by creating an SvJs class instance and calling some of its methods.

```
const svg = new SvJs();  
svg.addTo(document.getElementById('container'));
```

This is the extent to which we will be working with classes; writing them won't be part of this book.

Idiosyncrasies and Other Features

JavaScript offers plenty of other features, including closures, error catching, proxies, promises, and more, but I can't cover them all here. What I will point out here are some of the oddities – or quirks if we're being kinder – associated with JavaScript that may confound coders coming from another language.

Null and NaN Weirdness

We learned previously that `null` is essentially the absence of information. Isn't it strange then that `null` is also considered an object? We can confirm this by using the `typeof` check.

```
typeof null;
-> object
```

Despite this, `null` is not considered an instance *of* an object. We can confirm this by using the `instanceof` check.

```
null instanceof Object;
-> false
```

`NaN` (not a number) also presents us with some quantum-level weirdness that would have Schrodinger's cat perk up its ears. It is apparently not a number but equal to a number at the same time. And, bizarrely, not even equal to itself.

```
typeof NaN;
-> Number
```

```
NaN === Number;
-> false
```

```
NaN === NaN
-> false
```

Secret Casting

JavaScript does some things behind the scenes that programmers from stricter languages may not always appreciate. It is, for example, notoriously loose with data types and will convert variables of different types into matching types to suit the situation (the process of converting a data type is also known as casting it). Let's play with some numbers as an example.

```
1 == 1;  
-> true
```

```
1 == "1";  
-> true
```

How can a number equal a string? Well, it can't. JavaScript is casting the string as a number to make sense of the expression. It is essentially coercing the string data into number data (also known as type coercion). That's why we're always better off using the strict equality operator in JavaScript, something that doesn't exist in many other languages.

```
1 === "1";  
-> false
```

This kind of casting also occurs when we try to add numbers to strings, but it works in the opposite direction; the numbers in this case are converted to strings.

```
1 + "2";  
-> "12"
```

```
"2" + 3;  
-> "23"
```

To avoid the pitfalls of this kind of implicit type coercion, functions like `parseInt()`, `parseFloat()`, and `toString()` can be used to re-cast or explicitly convert variables to the data type you want to work with.

Semicolon Uncertainty

I mentioned at the start of the chapter that you can write JavaScript without semicolons if you wish, but that in some edge cases, it can result in unexpected behavior. Here's one such edge case.

```
return  
1 + 2  
-> undefined
```

Spacing doesn't matter in JavaScript, so what is going on here? Why isn't the expression returning 3? Well, here the Automatic Semicolon Insertion feature is being invoked under the hood. The actual code being evaluated is this:

```
return;  
1 + 2;
```

To avoid this, either familiarize yourself with such edge cases or just use semicolons.

Summary

We've covered a lot in this chapter:

- Basic rules of JavaScript syntax
- Values, such as strings, numbers, and booleans
- Variables and the different ways to declare them
- Arithmetic, comparison, and logical operators
- Conditional statements and control flow
- While loops, for loops, and forEach loops
- Functions and scope
- Arrays and array methods
- Objects and object properties, and alternatives to global variables
- Some of JavaScript's quirkiest features

CHAPTER 2 A PROGRAMMING PRIMER

I've assumed next to no coding knowledge, so if that describes your starting point and you've still kept up, well done!

In the next chapter, we'll explore the main functionality of the SvJs library and get comfortable creating basic shapes, lines, and colors.

CHAPTER 3

All About SVG

With the basics of JavaScript under your belt, you're now ready for SvJs, which you can think of as your generative brush and palette.

In this chapter, we'll cover setting up our SVG canvas; how to work with line and color; how to create a range of simple shapes; how to add text and titles; how to define and reuse elements; how to use groups, gradients, and patterns; and how the SVG viewBox and viewport work; and in the process, we'll learn the core functions of the SvJs library and how it relates to the SVG spec.

The Parent SVG Element

Before we can create anything visually, we need to load our library and set up the parent SVG element. The parent SVG, or the SVG document fragment as it's also known, is that SVG element whose parent element (if it has one) is not in the SVG namespace. An example would be an SVG attached to an HTML DOM node.

```
<div id="container">  
  <svg></svg>  
</div>
```

This is worth mentioning only because SVGs can contain other SVGs; in these cases, only one of those can be considered the parent SVG element. You might also see this referred to as the root element.

Let's reopen the JavaScript template file we set up in Chapter 1 (it should be located in `sketches/00-template/sketch.js`) and add the following line to it:

```
// Parent SVG.  
const svg = new SvJs();
```

The new `SvJs()` constructor, if called without any arguments, creates an SVG element. It can also be used to create any other SVG element by passing in the name of that element as a string.

```
// Using the SvJs constructor to create a circle element.  
const circle = new SvJs('circle');
```

To add our parent SVG element to our web page, we use the `SvJs.addTo()` method. In this case, we'll pass in the `<div>` with the id of container that we set up earlier in our HTML (see Chapter 1). Add the following to the template sketch:

```
const container = document.getElementById('container'); svg.  
addTo(container);
```

Most functions in `SvJs` are chainable, meaning you can chain them together using dot notation, like so:

```
methodOne().methodTwo();
```

The new `SvJs()` constructor method is no exception, so our new parent SVG element can actually be immediately attached to a DOM element once instantiated, by appending the `addTo()` method to it. This creates a method chain (think of the dot as the link). And if we pass the `getElementById()` DOM method as an argument instead of assigning it to a variable, we can condense our code down to a single line:

```
// Parent SVG.  
const svg = new SvJs().addTo(document.  
getElementById('container'));
```


Concision doesn't always beat clarity however, so if this one-liner is a little too compressed for your liking, keep your template code as is, no need to replace it with the aforementioned.

The Viewport and ViewBox

The viewport refers to the visible portion of our SVG element. It is defined implicitly, via the use of width and height attributes.

One of the interesting things about the SVG format is its flexibility with regard to its workspace, that is, the areas in which it can contain content. An SVG may contain much more besides what is visible at any one time through the viewport. This opens up the possibility of working with sprites, artboards, and other offscreen assets.

Because we'll be working with a square SVG viewport, our width and height will be the same, so we can use a single variable to store this value. We'll call this `svgSize`. We don't know in advance what the screen size or orientation will be, so hard-coding a value wouldn't be wise. Instead, we can check the browser window dimensions by querying `window.innerWidth` and `window.innerHeight` and set `svgSize` to the smaller of the two values. This will ensure a square (or 1:1) aspect ratio. The ternary operator that we covered in the previous chapter is particularly useful for these kinds of checks. Include it as follows in the template to initialize the `svgSize` variable:

```
// Viewport size (1:1 aspect ratio).  
const svgSize = window.innerWidth > window.innerHeight ?  
window.innerHeight : window.innerWidth;
```

The `viewBox` is another important top-level SVG attribute. It is related to the viewport but has more powerful (and more complex) capabilities. The `viewBox` allows us to define the position and dimensions of our viewport in user space.

What does this mean exactly? Well, you can think of user space as unitless, or unit-independent, space. A value of 100 might refer to pixels, millimeters, meters, or any other unit. It will default to pixels, but the point of the `viewBox` is that it can free us from thinking in absolute units. We can work with our own internal coordinate system without worrying about the actual dimensions of the device used to render the SVG.

Personally, I like working with a width and height of 1000, as I find it intuitive to divide up a canvas that way. Those 1000 units could later be squeezed into a square 500px wide, or splashed across a billboard the size of a bus.

The first two values of the `viewBox` define its starting coordinates (known as `min-x` and `min-y`), and the second two define the width and height. We'll keep things simple by defining the `viewBox` as `0 0 1000 1000`.

If the `viewport` and `viewBox` don't share the same aspect (i.e., width to height) ratio, the `preserveAspectRatio` attribute can be used to specify how the browser should display the SVG. This can get complex quite quickly, which is why I've already ensured our `viewport` and `viewBox` will share the same 1:1 (i.e., square) aspect ratio. For now, just be aware the `preserveAspectRatio` attribute exists, should you wish to facilitate more elaborate setups in future.

Sometimes to best understand how something works, you need to visually interact with it. With that in mind, I built a little CodePen sketch (or "pen" as they're called) demonstrating the effect of adjusting the main `viewBox` values, so you can see how they can be used together to scale and crop the content visible within the SVG viewport. You can find the pen at davidmatthew.ie/generative-art-javascript-svg#viewBox

At this point, we have the values we need for our `viewBox` and `viewport`, but we haven't yet learned how to put them to use.

Setting and Getting Values

Every SvJs instance, whether it's the parent SVG or one of its child elements, has a `set()` method. This method allows us to set the values of an element's attributes.

How do we know what attributes an element has? Well, we'd consult (i.e., google) some official documentation. The source I'd recommend is the Mozilla Developer Network's SVG element reference, available at developer.mozilla.org/en-US/docs/Web/SVG/Element. Use it for reminders or for when you do your own independent explorations. And while we're at it, the full documentation for the SvJs library is available at npmjs.com/package/svjs (or at the related GitHub repository); if you'd prefer, have that to hand for a more structured definition of each method.

Getting back to setting values, the `set()` method accepts an object as its argument and takes the following format:

```
svg.set({ attribute1: value1, attribute2: value2, attributeN:
valueN });
```

If you're setting a large number of properties at once, you might consider adjusting the formatting for the sake of readability.

```
svg.set({
  attribute1: value1,
  attribute2: value2,
  ...
  attributeN: valueN
});
```

It's up to you however; either way works.

The attributes of note for our parent SVG are the width and height (which constitute the viewport) and the viewBox. We can now add the following code to our template to set these attributes:

```
svg.set({ width: svgSize, height: svgSize, viewBox: '0 0 1000 1000' });
```

If you run the live-server command and navigate to our template folder, you'll just see a blank page at this point. This is because our SVG canvas doesn't contain anything to distinguish it from the background. We'll address this shortly by creating a rectangle to act as a background layer/canvas.

Just as we can `set()` values, we can also `get()` them. It's unlikely you'll find yourself doing this too often, but it's good to be aware the functionality exists. The following code, for example, outputs our `viewBox` value to the console:

```
console.log(svg.get('viewBox'));
```

Fetching attribute values via the `get()` method can be useful when working with event listeners, or in other cases where the values are being updated dynamically.

Quicker Element Creation

If we had to call the new `SvJs` constructor and the `addTo` method each time we created a new SVG element, it might get a little tiresome after a while. This is where the `SvJs` `create()` method comes in, as it combines these two steps into one. To create an element and automatically append it to our parent SVG, we'd simply write

```
svg.create('elementName');
```

When creating an element, it's usually best to assign it to a variable in the process, unless you're sure you won't need a reference to it again later. You might therefore write the aforementioned as

```
const variableName = svg.create('elementName');
```

The `create()` method is chainable and can be called by child elements too.

Lines and Shapes

The SVG spec defines six primitive shape types:

- Circles
- Ellipses
- Lines
- Polygons
- Polylines
- Rectangles

As we need to set up a background, let's start with the rectangle.

Rectangles and Squares

A rectangle is a `rect` in the SVG spec, so to create one, we'd write

```
svg.create('rect');
```

Next, we'll chain the `set()` method, focusing on the following five attributes of the `rect` element: `x` and `y` to set the position of its top left corner; `width` and `height` to define its size; and `fill` to define its color.

CHAPTER 3 ALL ABOUT SVG

As we want a square rectangle (as a square is technically just a special kind of rectangle), we just need to set our width and height to the same value.

Add the following code to our template:

```
// Background.  
svg.create('rect').set({  
  x: 0, y: 0, width: 1000, height: 1000, fill: '#181818'  
});
```

Note that we're defining our background's position and dimensions relative to the `viewBox`, not the `viewport`. This keeps our values consistent, as we don't know in advance what size our `viewport` will be. If you run `live-server` now, you should see a square canvas just a subtle shade darker than the surrounding background.

This will suffice for our template. In case you've missed anything earlier or just want to ensure your code is in order, here's the template `sketch.js` file in full:

```
import { SvJs } from '../..node_modules/svjs/src/index.js';  
  
// Parent SVG.  
const svg = new SvJs().addTo(document.  
  getElementById('container'));  
  
// Viewport and viewBox (1:1 aspect ratio).  
const svgSize = window.innerWidth > window.innerHeight ?  
  window.innerHeight : window.innerWidth;  
svg.set({ width: svgSize, height: svgSize, viewBox: '0 0 1000  
1000' });  
  
// Background.  
svg.create('rect').set({  
  x: 0, y: 0, width: 1000, height: 1000, fill: '#181818'  
});
```

Any further code we write will be specific to the sketch we're working on. Make sure you've saved your changes before copying the 00-template folder and its contents, renaming it to 02-basic-shapes.

First Strokes

There are two important attributes that can be used on practically every graphical SVG element (i.e., elements that are intended to appear on screen). These are an element's **fill** and **stroke**. As we've already seen, the **fill** defines an element's interior color (or background). The **stroke**, on the other hand, defines the color of an element's outline (or border).

If you don't define a **stroke**, none will appear. And if you do define one, its **stroke-width** will default to 1. The **stroke-width** attribute is important to highlight for another reason: it's an example of a hyphenated attribute. We can't use hyphens when setting attributes in JavaScript, as they will be interpreted as subtraction operators.

We have two options to deal with this: we can turn them into strings, that is, write 'stroke-width' instead of stroke-width, or we can replace hyphens with underscores, that is, stroke-width becomes stroke_width. The latter option is facilitated by SvJs, and it's the solution I prefer; it's faster and looks less out of place than having stringified object properties mixed with others that aren't strings.

Enough with the theory for now – let's create something! The following sketch is a Josef Albers-inspired color illusion and shows that with just plain rectangles, it's possible to create a somewhat interesting composition (illustrated in Figure 3-1).

```
import { SvJs } from '../..node_modules/svjs/src/index.js';

// Viewport size (1:1 aspect ratio).
const svgSize = window.innerWidth > window.innerHeight ?
window.innerHeight :
window.innerWidth;
```

CHAPTER 3 ALL ABOUT SVG

```
// Parent SVG.
const svg = new SvJs().addTo(document.
getElementById('container'));
svg.set({ width: svgSize, height: svgSize, viewBox: '0 0 1000
1000' });

// Background.
svg.create('rect').set({
  x: 0, y: 0, width: 1000, height: 1000, fill: '#181818'
});

// Main orange square.
svg.create('rect').set({
  x: 150, y: 200, width: 700, height: 600, rx: 15, fill:
  '#e56411',
  stroke: '#fff', stroke_width: 30, paint_order: 'stroke'
});

// Blue rectangle.
svg.create('rect').set({
  x: 650, y: 200, width: 200, height: 600, rx: 15, fill:
  '#69969f'
});

// Smaller orange rectangle.
svg.create('rect').set({
  x: 200, y: 425, width: 600, height: 150, rx: 20, fill:
  '#b84b08'
});

// Yellow rectangle.
svg.create('rect').set({
  x: 325, y: 200, width: 175, height: 600, fill: '#fed322'
});
```



```
// Purple rectangle.
svg.create('rect').set({
  x: 500, y: 200, width: 175, height: 600, fill: '#49283c'
});
```



Figure 3-1. *A Josef Albers-inspired color illusion*

Do the smaller rectangles on the right and left look like the same shade of orange? Assuming this illusion was successful, you might have some trouble settling on a definite answer. But by browsing the code, you can easily verify that they are not only the same shade, but the same shape!

What gives the rectangles those rounded corners is the `rx` attribute, which means its radius on the x axis. If the corresponding `ry` attribute isn't defined, it's assumed you want the same value (resulting in a uniformly curved corner).

You also might have spotted another hyphenated attribute in the preceding code, namely, `paint-order`. This determines in what order our fill and stroke are rendered for a given element.

The default order has fill come first (i.e., the fill is beneath so it will get partly painted over by the stroke), but sometimes it can be useful to have stroke come first, as only then will we see the full fill dimensions painted (remove `paint_order: 'stroke'` from the code and you'll see what I mean).

Circles and Ellipses

The circle is the simplest (and arguably the most perfect) of all shapes. We define the position of its center via `cx` and `cy` attributes and its radius via the `r` attribute.

```
// A simple circle example.
svg.create('circle').set({ cx: 50, cy: 50, r: 25 });
```

In the next sketch, let's shift things up a gear with a `for` loop. We're going to play with the effect of layering semitransparent circles of different sizes over each other (shown in Figure 3-2). Copy our template folder again and rename it to `03-circle-overlay-loop` or something similar (the names of our folders don't really matter provided we maintain a modicum of organization).

Beneath where we defined our background, include the following code:

```
// Circle overlay loop.
for (let i = 1; i <= 6; i += 1) {

  // Vary the radius, and the two vertical centre points.
  let r = 50 * i;
  let cx = 500;
  let cy1 = 800 - r;
  let cy2 = 200 + r;
```

```
// Create the blueish circle set.
svg.create('circle').set({
  cx: cx, cy: cy1, r: r, fill: '#99eeff', fill_opacity: 0.1
});

// Create the greenish circle set.
svg.create('circle').set({
  cx: cx, cy: cy2, r: r, fill: '#aaffee', fill_opacity: 0.1
});
}

// Create a subtle outline.
svg.create('circle').set({
  cx: 500, cy: 500, r: 320, fill: 'none',
  stroke: '#aaffee', stroke_width: 2, stroke_opacity: 0.1
});
```

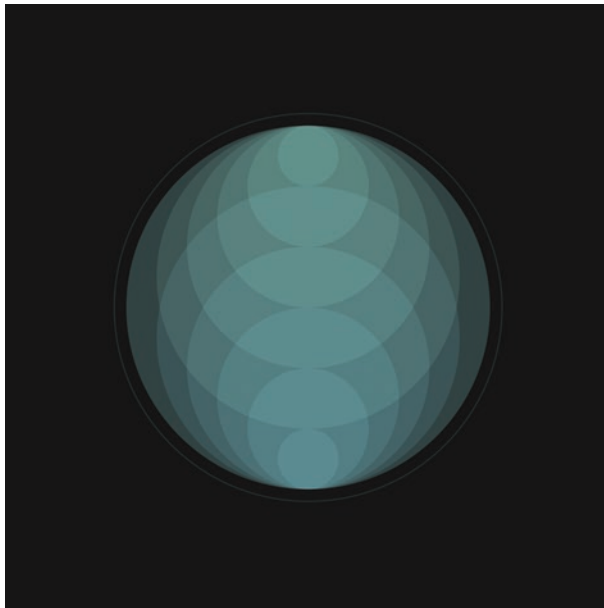


Figure 3-2. *The result of our circle overlay loop*

What we've done here is vary the vertical position of the `cy` attribute and increase the radius `r` for two sets of circles each time the loop runs. The circle sets are differentiated by their color (slightly) and initial positions. A final circle outside the loop provides a subtle outline to frame the circle sets.

Ellipses are similar to circles but offer two radii attributes – `rx` and `ry` – instead of the single `r`. In the first generative sketch from Chapter 1, we saw the ellipse in action. Here's the relevant snippet:

```
// Create our ellipse.
let ellipse = svg.create('ellipse');
ellipse.set({
  cx: center,
  cy: center,
  rx: radiusX,
  ry: radiusY,
  ...
});
```

Lines, Polylines, and Polygons

If you need to draw a simple straight line from one point to another, you'd use the `line` element. It requires two sets of coordinates: `x1` and `y1` to define the initial position and `x2` and `y2` to define the endpoint. It also requires the stroke to be defined (as it is a line after all).

```
// The humble line.
svg.create('line').set({
  x1: 10, y1: 10, x2: 90, y2: 90, stroke: '#333'
});
```

This is a good opportunity to showcase some other stroke-related attributes that can alter the appearance of a line. The first is `stroke-linecap`. This defines how the endpoints of a line behave. The default value, `butt`, limits the line length to its endpoints. A value of `round` will close off the line with a semi-circle at each end, whereas a value of `square` will close it off with – you guessed it – a square. Figure 3-3 shows the different values, with vertical lines on either side to delineate where the values come into effect.

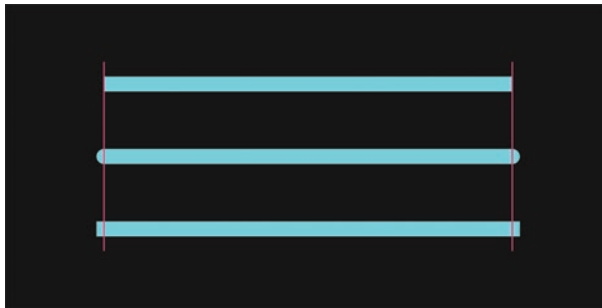


Figure 3-3. *Values of butt, round, and square (top to bottom)*

The other attribute I want to highlight is `stroke-dasharray`. If, rather than a solid line, you would prefer a stroke consisting of dashes, this is what you would use. More specifically, the `stroke-dasharray` sets the pattern of dashes – and the gaps between them – with which to paint a stroke.

If we set a single value, say 20, that value will be used for the length of the dash and the length of the gap. If we set two values, the first is used for the dash and the second for the gap. If however we set three values, the first is used for the dash, the second for the gap, and the third for the next dash. The values are then cycled through again, creating a repeating pattern.

There isn't a strict upper limit on the amount of values you provide, so there is scope to get quite creative here.

```
// Setting an irregular stroke-dasharray.
svg.create('line').set({
  ...
  stroke_dasharray: 10 30 60
});
```

In Figure 3-4, from top to bottom, we see the following stroke-dasharray values (all relative to a viewBox width of 1000):

- 20
- 40 20
- 10 30 60
- 50 100 200 100



Figure 3-4. *Various stroke-dasharray values*

If you want to connect one straight line to another, you'd use a `polyline` element. This takes a `points` array of space-separated (or comma-separated) values to define each `x` and `y` coordinate. The array of points needs to be converted to a string before being set as the attribute value. I show how to do this in the following example using the `join()` array method. The result is shown in Figure 3-5.

```
// An array of points.
let pointsArray = [
  100, 250, 250, 250, 285, 175, 325, 325, 390, 50, 450, 400,
  500, 200, 515, 250, 900, 250
];

// A polyline element, mimicking a heartbeat.
svg.create('polyline').set({
  points: pointsArray.join(' '),
  stroke: '#6df876',
  fill: 'none',
  stroke_width: 10,
  stroke_linecap: 'round',
  stroke_linejoin: 'round'
});
```



Figure 3-5. *A polyline element in action*

I've also used the `stroke-linejoin` attribute in the preceding example; this defines the shape to use at point where each line meets the next.

A polygon element is similar to a polyline element but is used for closed shapes, that is, where the start and end points connect. In Figure 3-6, we see an arrangement of polygons that might be familiar to gamers of a particular platform.

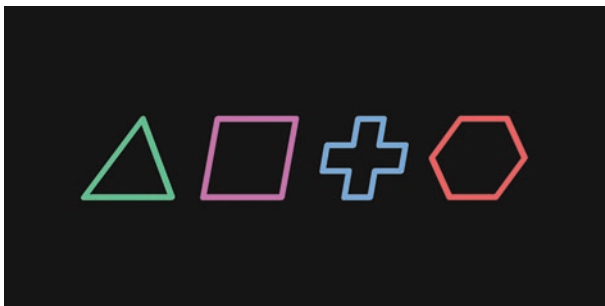


Figure 3-6. *Polygons for the players*

Manually creating shapes with either the `polygon` or `polyline` elements isn't something we'll focus further on for now, as it can be quite tedious. There are better ways to generate shapes that we'll cover in a later chapter.

Text and Titles

As SVG is a plain text format, it's no surprise that it supports textual elements. To create some text, we use the usual `svg.create()` method. At a minimum, we need to define the `x` and `y` coordinates of where we want it to appear (relative to the bottom left corner of the text's bounding box), its color, and its content.

```
// Creating text.
let text = svg.create('text').set({
  x: 20, y: 30, fill: 'white'
});
```

So how do we define the textual content if the content isn't itself an attribute? It's not a child node either, so creating a string and appending it wouldn't work. Instead, what we need to do is use the SvJs `content()` method.

```
// Using the content() method to insert text.
text.content('I have an unhealthy obsession with SVG.');
```


What if we wanted to get a bit more elaborate with our text and style them with some web fonts? There are a few ways we could do this, but I'll stick with the simplest and pull in some Google fonts via our HTML. Let's use this as the basis of a quick sketch; copy our 00-template folder and call it 04-chalkboard-gag.

If you've ever watched *The Simpsons*, you'll know that the beginning of most episodes features a chalkboard gag, where Bart is shown scrawling lines on a chalkboard as punishment for his errant behavior. One of my favorites is featured in the following sketch.

What we need to do first is include the following `<link>` in our HTML, so switch over to the `index.html` file and add the following line to the `<head>` section. This calls in a Google font called Mynerve, which has the handwritten look we're going for.

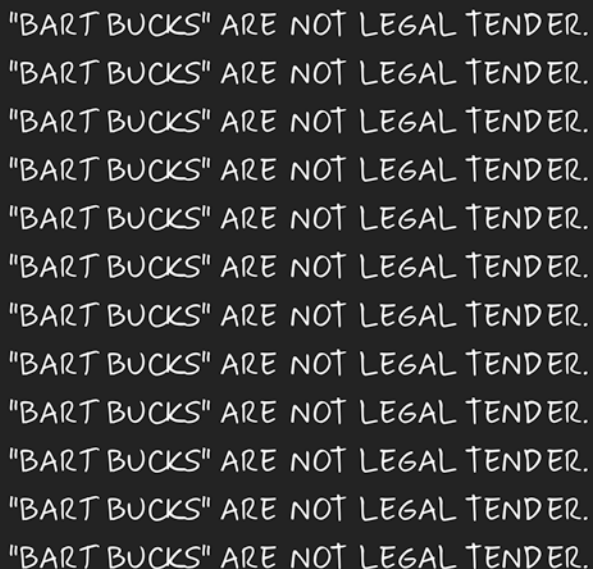
```
<link href="https://fonts.googleapis.com/css2?family=Mynerve
&display=swap"
rel="stylesheet">
```

If Bart had the option of running a for loop to minimize the repetitive nature of his punishment, I'm sure he would have taken it. Let's do this on his behalf (and cheat Skinner out of some satisfaction).

Add the following code below where we declare our background. Some points to note: we're calling the `toUpperCase()` string method to capitalize the content; this is a useful method to know, as is the related `toLowerCase()` method. The `font_size` of 52 is used to optimally fill the `viewBox` width, and incrementing the `y` position on each iteration by 80 is what pushes the lines down our chalkboard. We have enough space to do this 12 times, which is why we limit incrementation to below 960 ($960 \div 80 = 12$). Another iteration would overflow the viewport.

The output is shown in Figure 3-7.

```
// The line to use for the gag.  
let line = '"Bart Bucks" are not legal tender.';  
line = line.toUpperCase();  
// Run a loop, creating 12 (960 / 80) lines of text.  
for (let i = 0; i < 960; i += 80) {  
  let text = svg.create('text');  
  text.content(line);  
  text.set({  
    x: 20,  
    y: 80 + i,  
    fill: '#fff',  
    font_size: 52,  
    font_family: 'Mynerve'  
  });  
}
```



"BART BUCKS" ARE NOT LEGAL TENDER.
"BART BUCKS" ARE NOT LEGAL TENDER.
"BART BUCKS" ARE NOT LEGAL TENDER.
"BART BUCKS" ARE NOT LEGAL TENDER.
"BART BUCKS" ARE NOT LEGAL TENDER.
"BART BUCKS" ARE NOT LEGAL TENDER.
"BART BUCKS" ARE NOT LEGAL TENDER.
"BART BUCKS" ARE NOT LEGAL TENDER.
"BART BUCKS" ARE NOT LEGAL TENDER.
"BART BUCKS" ARE NOT LEGAL TENDER.
"BART BUCKS" ARE NOT LEGAL TENDER.
"BART BUCKS" ARE NOT LEGAL TENDER.

Figure 3-7. The “Bart Bucks” chalkboard gag

Titles are another type of text we can use. Unlike a text element, a title element doesn't need to be positioned; it is intended to be nested within an element as a description. It works in the same way an HTML title attribute works; it helps with accessibility and appears as a tooltip on hover. It can be used to title the parent SVG element, or any of its child elements.

```
// Create a title for a circle element.
let circle = svg.create('circle');
let circleTitle = circle.create('title'); circleTitle.
content('I get around-round, I get around.');
```

A title element is suitable for shorter descriptions; for more extended descriptions, the desc element is recommended. This doesn't render and is purely for accessibility purposes, and is created in exactly the same way as a title element.

If you want to add extra information about the SVG file itself, the metadata element can be used. An example would be embedding RDF (Resource Description Framework) data, or application-specific data. You'll often see software like Inkscape or Illustrator embed their own metadata in this manner.

Definitions

If we want to define a graphical or other container element for use in our SVG but don't want that element itself to appear (or if we want it to appear only when referenced), we would nest it within the defs element, which stands for definitions. The defs element can improve the overall accessibility of an SVG and also better help organize its content.

We'll rely on the functionality provided by the defs element in the upcoming sections on gradients, patterns, and other reusable elements.

```
// Defining a defs element.
let defs = svg.create('defs');
```

Gradients

A gradient is a gradual transition from one color state to another. In SVG, gradients can support a multitude of these different color states (or “stops” as they’re called). Most often, you’ll see gradients of the two-stop variety, but occasionally you’ll come across one boasting a rainbow-esque array of different colors. And speaking of rainbows, the sky itself is a great source of gradient inspiration (gradienspiration?). Think of the dim light at dusk where soft yellows transition to light blues, or the brilliant blaze of a sunset where purples, reds, and oranges collide.

Gradients can be of two varieties: linear (a color transition that proceeds from one area to another in a straight line) and radial (a color transition that emanates outward in all directions from a focal point).

How would we create such gradients in SvJs? It turns out it would be a very verbose process using just the `create()` method, and there’s a couple of reasons for this:

- Gradients are defined as stand-alone elements within the `defs` section rather than treated as purely presentational element attributes. The upside of this is that a single gradient can be shared among multiple elements, but the downside is we’ve got a couple of extra steps to begin with.
- For each gradient color, a stop element must be defined and nested within the gradient element to define the color and its offset (or position). This adds another series of steps.

To remedy the aforementioned, SvJs offers a dedicated `createGradient()` method that significantly speeds up the process of constructing a gradient. Its syntax is as follows:

```
createGradient(id, type, colours, rotation)
```

With the first argument, we assign an id to the gradient so that it can be referenced or called upon later by an element. With the second argument, we specify the gradient type, that is, linear or radial. With the third argument, we pass in an array of color values. And with the final argument, we specify the angle of rotation, which is 45° by default (meaning it “shines” from the top left to the bottom right). This last argument is optional and is only applicable to gradients of the linear variety.

The `createGradient()` method can only be called by the parent SVG, as behind the scenes it checks whether a `defs` element is already defined within the SVG, and if not, it creates it and appends the gradient to it.

Here’s how we might create a simple three-stop linear gradient and apply it to a `rect` element.

```
// Create the gradient with an id of 'grad'.
svg.createGradient('grad', 'linear', ['red', 'orange',
'yellow'], 90);

// Apply the gradient to the fill of the rect via a url
reference.
svg.create('rect').set({
  x: 0, y: 0, width: 400, height: 400, fill: 'url(#grad)'
});
```

As you can see, the shape we want the gradient applied to references the gradient id as an internal url. In the preceding example, we’ve applied it to the `fill` attribute, but it could also have been applied to the `stroke`.

In Figure 3-8, we see an example of three linear gradients and their radial equivalents. The first gradient has two color stops, the second has three, and the third has four. As you can see, it’s easier to make linear gradients look smoother; with radial gradients, subtlety is always better, as transitions can seem relatively abrupt (or banded) compared to their linear counterparts.



Figure 3-8. *Linear gradients and their radial equivalents*

There are more ways to customize gradients that we won't get into here, but if you're interested, check out the MDN documentation for more on the `gradientSpreadMethod` and varying the focal points of radial gradients.

Patterns

Like gradients, patterns reside in the `defs` element to be referenced later rather than rendered directly. They have their own `viewport` (i.e., width and height) and optional `viewBox`, which can seem a little complicated at first, but it does allow for a greater degree of flexibility. Think of a pattern element as a miniature canvas to be replicated across a larger graphical element and these settings will make more sense.

To create a pattern, we can use the `createPattern()` shortcut method, which, as with the gradient method, automatically appends itself to a `defs` element and creates one if it doesn't exist. It also simplifies pattern creation in other ways under the hood that you don't have to worry about. It requires three arguments: the pattern id, the width, and the height.

To show how patterns work, we'll create a quick sketch – another optical illusion, but this time the focus won't be on the colors. Copy our 00-template folder and rename it to 05-optical-illusion.

In our `sketch.js` file below the code for the background, create a pattern with an id of 'illusion' and with a width of 100 and a height of 200.

```
// Create our pattern.
const pattern = svg.createPattern('illusion', 100, 200);
```

This simply initializes our pattern; we haven't placed anything within it yet, nor have we set up a shape to apply it to. We just have a pattern in the shape of a vertical rectangle.

In our first iteration of this sketch, I'll show you how a simple pattern gets applied to a shape. We'll get more elaborate with the pattern content afterward. Add the following code to our sketch:

```
// Create a white rectangle within the pattern.
pattern.create('rect').set({
  x: 10, y: 10, width: 80, height: 180, fill: '#eee'
});
```

Now that we have our simple pattern in place (consisting of a single white rectangle), it's time to apply it to a shape. For simplicity, let's apply it to another rect the full size of the SVG `viewBox`, like so:

```
// Apply our pattern to a rect the size of the viewBox.
svg.create('rect').set({
  x: 0, y: 0, width: 1000, height: 1000, fill: 'url(#illusion)'
});
```

You should now see something similar to Figure 3-9. Not especially interesting, but sufficient to showcase the basics.

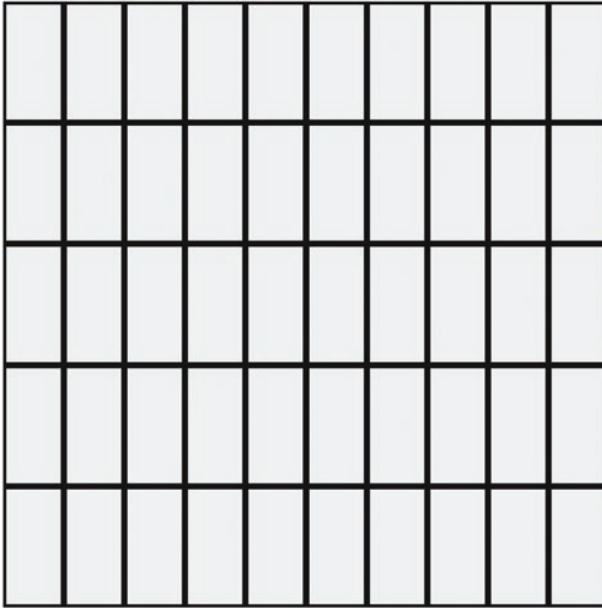


Figure 3-9. *A basic pattern*

In the next few steps, we'll create our optical illusion. First, comment out the code that created the white rectangle pattern (but keep the shape it's applied to).

```
// Create our pattern.  
const pattern = svg.createPattern('illusion', 100, 200);  
  
// Create a white rectangle within the pattern.  
// pattern.create('rect').set({  
//   x: 5, y: 5, width: 90, height: 190, fill: '#eee'  
// });  
  
// We'll place our new code here.
```



```
// Apply our pattern to a rect the size of the viewBox.
svg.create('rect').set({
  x: 0, y: 0, width: 1000, height: 1000, fill: 'url(#illusion)'
});
```

To create the base of our optical illusion, we need a total of eight rectangles. Four of them will be white squares arranged in a particular order, and four will be thin gray rectangles used as separators. We'll use a couple of for loops to create these.

First, the white squares:

```
// Create 4 x white squares within the pattern.
for (let i = 0; i < 4; i += 1) {
  pattern.create('rect').set({
    x: (i === 3) ? 20: i * 20,
    y: i * 50,
    width: 50,
    height: 50,
    fill: '#eee'
  });
}
```

The width, height, and fill merely repeat themselves on each iteration; the only attributes we're varying are the x and y coordinates. To do this, we're using the changing value of the i iterator. The y coordinate increases by 50 on each run, which is simple enough. The expression defining the x position is a little more complicated. What we're saying here is that we want x to increase by 20 on each iteration, *except* for the final one (where i === 3). For this, we want to set it to 20.

If you view the pattern at this point, you should see an undulating array of squares. It's not an optical illusion yet; it looks more like rudimentary pixel art. To complete the illusion, we need our separators.

```
// Create 4 x thin grey rectangles to separate the squares.  
for (let i = 0; i < 4; i += 1) {  
  pattern.create('rect').set({  
    x: 0,  
    y: 45 + (i * 50),  
    width: 100,  
    height: 5,  
    fill: '#666'  
  });  
}
```

Now when you hit save, you should see something like Figure 3-10.

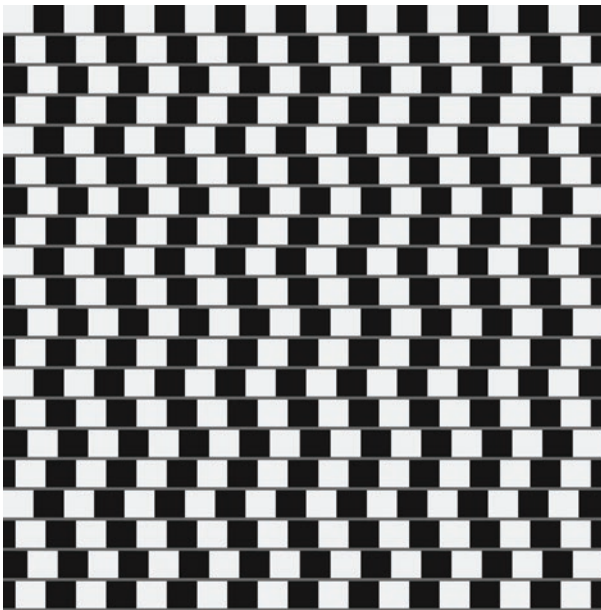


Figure 3-10. A repeating pattern creating an optical illusion

If the illusion worked, the gray lines should appear slightly slanted, particularly those you're not directly focusing on. Hopefully this will give you some sense of the power of patterns! We've touched on the main points, but they can be customized extensively with further parameters if needed.

Grouping and Reusing Elements

Grouping elements together can be done with the `g` element. There are a number of use cases for this. One, which applications like Inkscape have adopted, is to use the `g` element to support layer functionality. Another is to group elements together for the purposes of transformation (i.e., moving, rotating, or skewing multiple elements at once). And another is to simply group items together to better organize your SVG.

```
// Create a circle and add it to a group.
const circle = svg.create('circle');
const group = svg.create('g');
circle.addTo(group);
```

If you want to group elements together for later reuse (so that they are only displayed when referenced), you'd use the `symbol` element. Symbols, like patterns, can have their own viewport and `viewBox`. But they are called via the `use` element rather than applied to a fill or stroke.

```
// Create a symbol consisting of a circle and square.
const symbol = svg.create('symbol');
symbol.set({ id: 'mySymbol', width: 100, height: 100 });
symbol.create('circle').set({ ... });
symbol.create('rect').set({ ... });
```

```
// Using the symbol three times but varying its co-ordinates.  
svg.create('use').set({ href: '#mySymbol', x: 0, y: 0 });  
svg.create('use').set({ href: '#mySymbol', x: 200, y: 200 });  
svg.create('use').set({ href: '#mySymbol', x: 400, y: 400 });
```

Symbols don't have quite the same appeal outside of a declarative context, so going forward we won't be relying on them for our imperative-based creations. But there's plenty more to unpack if you want to explore further on MDN.

Summary

To recap, we've covered the following in this chapter:

- The core functions provided by the SvJs library
- How the SVG `viewBox` and `viewport` work
- Creating basic shapes like rectangles, circles, ellipses, polylines, and polygons
- Customizing stroke behavior and appearance
- Working with text and titles
- Creating gradients and patterns
- How definitions, groups, and symbols allow us to reuse elements and better structure our SVG

In the next chapter, we'll explore how we can combine regularity with randomness to create truly generative compositions.

CHAPTER 4

Randomness and Regularity

So far, all of our sketches (bar the first one) have lacked a certain ingredient usually considered foundational to the very genre of generative art: namely, randomness. When we surrender full control over the result of our sketches and leave certain elements to chance, it allows for variation, for exploration, and for *discovery*.

In this chapter, we'll learn how to randomize parameters like colors, sizes, and coordinates; how to randomly pick from predefined ranges and arrays; and how to work with different probability distributions. We'll also cover a popular generative technique that involves constructing regular grids and randomizing their contents. We'll then learn how to apply randomness to the construction of the grids themselves so that the underlying structure of our compositions can be varied.

Analogue and Digital Randomness

Techniques to incorporate randomness into art began long before the digital age. As far back as the fifteenth century, Leonardo da Vinci suggested that artists take inspiration from sources rich in random and suggestive detail, such as the arbitrary forms found on the surface of stained stones and dirty walls. Artists like Max Ernst took such prompts

quite literally; his frottage technique was borne of dropping paper at random on floorboards full of scrapes and scratches and making charcoal rubbings from the textures.

Contemporaries of Ernst, the early twentieth-century surrealists, also played with randomness. They developed a technique known as automatism, which aimed to eradicate full conscious control over an artwork's execution, allowing chance and the unconscious to act as the primary driving forces.

These examples, however, all draw upon the randomness found in the analogue world. In the more deterministic world of the digital, where ones and zeros reign, true randomness is harder to find. It must instead be simulated.

The good news is that this is more of an academic point than a practical obstacle. Simulated randomness (or pseudo-randomness as it's referred to in the field of computer science) is more than enough for our needs. And it's very easy to demonstrate this too, thanks to JavaScript's built-in `Math.random()` function. Typing the function into your browser console and hitting return should provide sufficient assurance that the 17-decimal floating-point number it throws back at you is unlikely to repeat itself anytime soon. Random enough, right?

```
Math.random();  
-> 0.32846662956382255
```

The limitation of this function is that it only returns a number between 0 and 1. Granted, we can easily multiply the output by whatever factor we want, but what if we needed a number within a specific range? Or wanted to choose an item at random from an array?

The SvJs `Gen.random()` Function

Thankfully the SvJs library comes armed with some extras: specifically, an optional module called `Gen` that contains a number of useful functions for generative artists. One of these is the `Gen.random()` function.

To load the `Gen` module, we need to add it to our import statement at the top of our sketches. Modify the template sketch to include this as the first line (replacing the previous import statement):

```
import { SvJs, Gen } from '../..//node_modules/svjs/src/
index.js';
```

Now we can call `Gen.random()` and any of the other `Gen` methods (more of which we'll cover later). When called without any arguments, `Gen.random()` behaves just like `Math.random()`; it returns a floating-point number between 0 and 1.

```
Gen.random();
-> 0.5682831319665758
```

If, however, we supply some arguments, it suddenly becomes a lot more useful. Here are some examples:

```
// Return an integer between 50 and 100.
Gen.random(50, 100);
-> 87
```

```
// Return a floating point number between 10 and 20.
Gen.random(10, 20, true);
-> 17.98188644344106
```

```
// Return a random item from an array.
let rainbow = ['red', 'yellow', 'pink', 'green', 'purple',
'orange', 'blue'];
Gen.random(rainbow);
-> 'purple'
```

As you can see, if we provide two numbers, the `Gen.random()` function will treat them as the lower and upper thresholds of a range and return a number within that range. An integer is returned by default, but if we set the third argument to `true`, it will return a floating-point number. It will also return a floating-point number if we supply no arguments at all (as in our first example), or if the difference between the first and second arguments is less than or equal to one.

If we pass in an array as the sole argument, an item is randomly selected from that array. I purposely used colors in the example (apologies if the “Sing a Rainbow” song got stuck in your head), as this functionality is particularly useful when selecting colors from a predefined palette.

Elements Everywhere All at Once

Let’s create a sketch and put `Gen.random()` to work. Copy the 00-template file as we usually do and rename it to something like 06-elements-everywhere.

Below our background code, initialize a variable to store a random iteration count, which will determine how many times our `for` loop (which we’ll set up after) will run.

```
let iterations = Gen.random(500, 1000);
```

Our iterations will be an integer anywhere between 500 and 1000. With that in place, let’s write the loop. We’ll keep it simple initially and just use it to create small circles. Each circle will be given a random position within the dimensions of the `viewBox` (0 to 1000), and its radius will be between 1 and 10. This should create an effect not too dissimilar to the spattering of paint (illustrated in Figure 4-1).

```
for (let i = 0; i < iterations; i += 1) {
  svg.create('circle').set({
    cx: Gen.random(0, 1000),
```



```
    cy: Gen.random(0, 1000),  
    r: Gen.random(1, 10)  
  });  
}
```

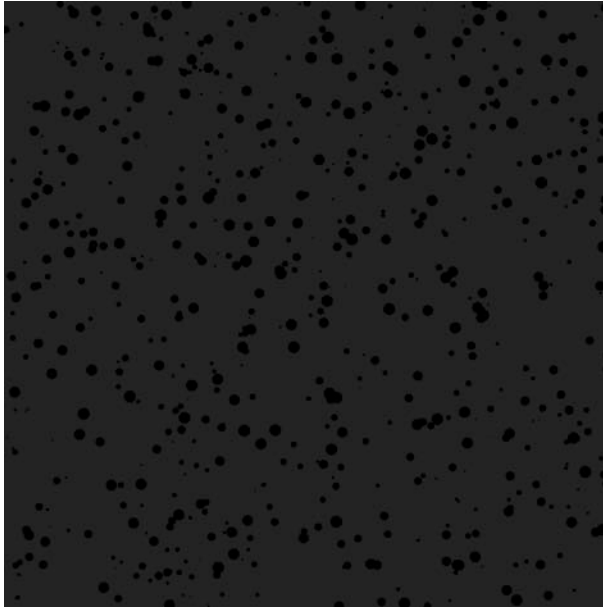


Figure 4-1. *A random spattering of circles*

Varying Color and Opacity

At this point, we have colorless spatters randomly speckling the canvas. Let's change this up by varying the hue and opacity of the fill. We'll do this with the `hsl()` function, which is something we'll be drawing upon quite a lot. Its syntax is as follows:

```
hsl(hue saturation lightness / alpha)
```

The hue can be any number between 0 and 360. The saturation and lightness are percentage values, and the alpha component (which determines the opacity) can be either a percentage or a number between 0 and 1.

In our loop, add the following line below the radius (making sure to separate the lines with a comma):

```
fill: `hsl(${Gen.random(0, 360)} 80% 80% / ${Gen.  
random(5, 40)}%)`
```

Now our speckles have come a little more to life (see Figure 4-2). But why stick to circles? Why not throw some other elements into the mix and make it look a little less like a cosmic ball pit? In the next version of the sketch, we'll do precisely that. The complexity will ramp up considerably, so we'll step through it slowly.



Figure 4-2. *A more colorful spatter*

Varying Element Selection

Below our iterations variable, create an array called `elements` and populate it with a circle, a line, and a rect:

```
let elements = ['circle', 'line', 'rect'];
```

Next, delete all the existing code in our for loop, and replace it with the following line that will pick a random element from the `elements` array defined previously.

```
// Pick a random element.  
let element = Gen.random(elements);
```

In the next step, we'll initialize some variables with random values that we can later apply to our randomly selected element. In addition to our `x`, `y`, and `fill`, we'll also vary our `stroke` and `stroke-width`.

```
// Set up variables that we can use on any element.  
let x = Gen.random(200, 800);  
let y = Gen.random(200, 800);  
let fill = `hsl(${Gen.random(120, 240)} 80% 80% / ${Gen.  
random(5, 40)}%)`;   
let stroke = `hsl(${Gen.random(0, 120)} 80% 80% / ${Gen.  
random(5, 40)}%)`;   
let strokeWidth = `${Gen.random(1, 3)}`;
```

You'll notice here that I've constrained the `x` and `y` variables to a range of 200 to 800, which ensures our shapes remain within that region and don't cover the entire canvas. This is so that we end up something that more closely resembles a composition than an explosion.

Next we'll set up a `props` variable (a commonly used shorthand for properties). The idea is that we set our element properties in accordance with the element chosen. Given that each element has its own distinct syntax for certain attributes (e.g., a `rect` has an `x` and `y`, but a `circle` has a

cx and cy), we need to have a way of conditionally populating the props. There are a few ways we could approach this, but a switch statement lends itself particularly well to this situation. It allows us to populate the props variable on a case-by-case basis (quite literally, as you'll see).

```
// Initialise the properties variable.
let props;

// Populate the properties depending on the element chosen.
switch(element) {
  case 'circle':
    props = {
      cx: x, cy: y, r: Gen.random(1, 10),
      fill: fill, stroke: stroke, stroke_width: strokeWidth
    };
    break;
  case 'line':
    props = {
      x1: x, y1: y, x2: x + (Gen.random(-20, 20)),
      y2: y + (Gen.random(-20, 20)), stroke: stroke
    };
    break;
  case 'rect':
    props = {
      x: x, y: y, width: Gen.random(5, 25), height:
      Gen.random(5, 25),
      fill: fill, stroke: stroke, stroke_width: strokeWidth,
      transform: `rotate(${Gen.random(0, 360)} 500 500)`
    };
}
```

The one line here that perhaps calls for a little further explanation is the transform attribute included in the rect properties. This attribute can take either a translate, rotate, scale, or skew command as its argument,

along with a transform value and two other values that represent the transform origin. The origin is a set of x and y coordinates that define the point relative to which the transform occurs. For example, in the case of the rotate command, we are randomly rotating the rect relative to 500 500, that is, the center of the viewBox.

We have one line left to include in the loop, right after our switch statement. This is the line that actually creates the element and applies the props to it.

```
// Create the element and set its properties.  
svg.create(element).set(props);
```

When you run the sketch now, you should (hopefully) see something along the lines of Figure 4-3. Perhaps we weren't so successful in making it look less explosive (it looks like an eruption of confetti to me), but the different shapes and the addition of stroke values definitely add more in the way of variety and visual interest.

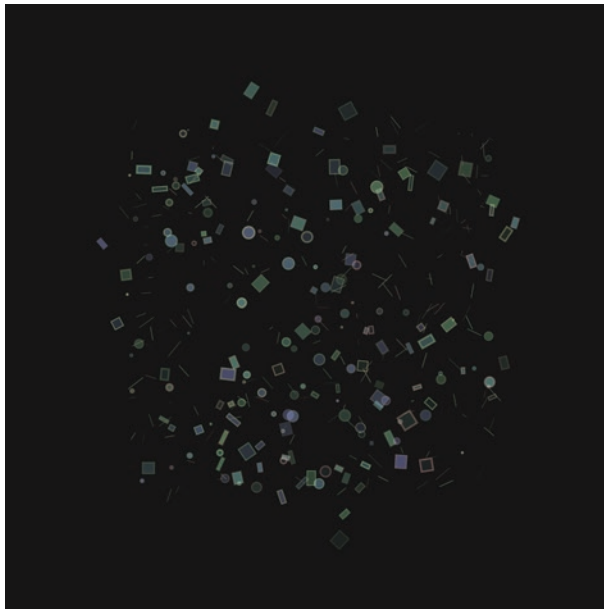


Figure 4-3. *A confetti-like scattering of random elements*

Regular Grids

Unrestrained randomness will only get us so far; we need techniques to tame and constrain it. One particularly prominent technique in the generative art world is to work with grids, which allow us to structure our randomness.

The Nested For Loop

The nested for loop consists of one loop running within another. Because grids are comprised of columns (along the x axis) and rows (along the y axis), the nested for loop comes in very useful in their construction. What we'll typically do is start along the y axis and create our rows, and for each row, we move from left to right along the x axis, filling out each cell within the column. This movement mimics how computers have historically drawn pixel data to their displays; start at the top left, move across, and work your way down.

The outer loop will therefore be our y axis, moving down. For each iteration of the y axis, the inner (nested) loop will move across, populating the x axis.

```
// Example nested loop structure.
for (let y = 0; y < height; y += 1) {
  for (let x = 0; x < width; x += 1) {
    // Code goes here.
  }
}
```

Let's create a sketch to better illustrate this. Copy and rename the template folder, renaming it to 07-regular-grids. Below the background, we'll set up a group element to act as our grid container.

```
// Create our grid container group.
let grid = svg.create('g');
```

Now we'll run our for loop, appending a `rect` element to the grid on each iteration.

```
// A nested loop to visualise the grid.
for (let y = 0; y < 700; y += 50) {
  for (let x = 0; x < 700; x += 50) {
    grid.create('rect').set({
      x: x, y: y, width: 40, height: 40, fill: 'none',
      stroke: '#eee'
    });
  }
}
```

The values we've used here aren't especially meaningful; we could have chosen a different increment value, iteration count, width and height for the square, etc. We can tidy up these values later. What you will notice if you run `live-server` is that the grid is positioned off-center, to the top left (as our `x` and `y` values initialize at 0). We could adjust the `x` and `y` values until the grid lines up with the center of the `viewBox`, but that would involve some tedious trial and error and would need to be adjusted each time we altered the grid. Instead, let's use the built-in `SvJs` `moveTo()` method.

The `moveTo()` method moves a given element by its center point to a new position. The new position is passed in as an `(x, y)` coordinate pair. This method is especially useful on a group element (where the center point isn't straightforward to calculate) but can also be used as a more intuitive alternative to the `transform: translate(x y)` syntax.

`SvJs` also offers a related `getCentre()` method (which the `moveTo()` method also utilizes internally) to retrieve an element's center coordinates, returned as an object with `x` and `y` properties. This is useful when you want to position one element inside another.

After the loop, call the `moveTo()` method as follows:

```
// Centre the grid within the viewBox.  
grid.moveTo(500, 500);
```

The grid should now line up nicely in the center of our `viewBox`, as shown in Figure 4-4.

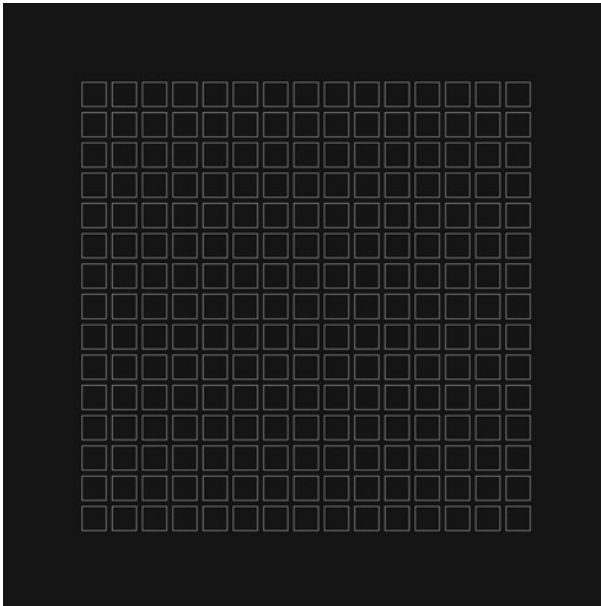


Figure 4-4. *A regular, centered grid*

A More Flexible Grid

So far we have hard-coded our grid values. By this, I mean we have used values directly to control the grid divisions, cell spacing, size, etc., rather than variables containing values. The latter approach allows for more flexibility.

Before our `for` loop, set and initialize the following variables. The values we'll use will reflect the grid we've already created; we'll alter these afterward.


```
// Set some grid-related variables.
let gridSize = 700;
let rows = 15;
let spacing = 5;
```

What we need next are variables that control the loop increment value and the size of each cell. These values, however, should be *derived* from those we've set previously, rather than worked out afresh each time we want to change the grid. With a little bit of basic math, here's how we'd do this:

```
let increment = gridSize / rows;
let cellSize = Math.abs(increment - spacing);
```

The `Math.abs()` function just ensures our `cellSize` is always a positive value, which can prevent errors if we go crazy with our rows and spacing values. Next, we need to incorporate these values into our loop. Adjust it as follows:

```
for (let y = 0; y < gridSize; y += increment) {
  for (let x = 0; x < gridSize; x += increment) {
    grid.create('rect').set({
      x: x, y: y, width: cellSize, height: cellSize,
      fill: 'none', stroke: '#eee',
    });
  }
}
```

Now we can freely adjust our grid variables, and the loop will respond accordingly. We could even randomize them and get a different grid on each refresh (as per Figure 4-5).

```
let gridSize = Gen.random(400, 800);
let rows = Gen.random(2, 20);
let spacing = Gen.random(5, 10);
```

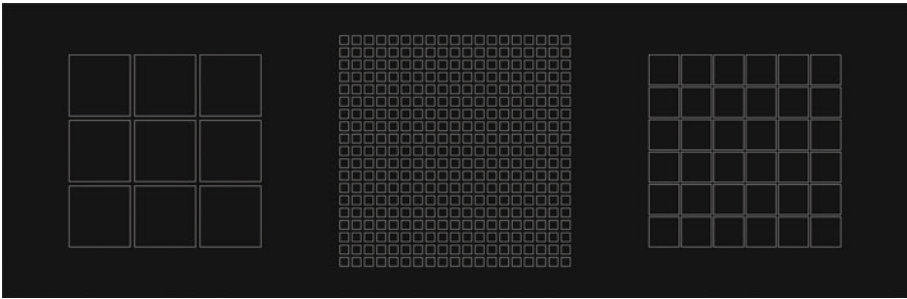


Figure 4-5. *Variations of our regular grid*

Clip Paths and Color Palettes

Now that we have our regular grid in place, we need to breathe some life into it. The changes we’re going to make are significant enough to warrant a new iteration of our sketch, so if you’d prefer to keep what we’ve done so far as a template, save and copy the existing sketch folder and rename it to something like 08-colourful-grids, which will give you a clue where we’re taking this.

Arrays of Colors

In our previous “Elements Everywhere” sketch, we used `Gen.random()` to generate our colors on the fly. But what if we wanted a particular color palette, or a collection of color palettes, so that we could randomly choose between them?

One popular approach is to store a pre-chosen palette as an array of color values, like so:

```
// Create a colour palette.
let palette = ['#5465FF', '#788BFF', '#9BB1FF', '#BFD7FF',
               '#E2FDFF'];
```

There are many sites out there, too numerous to name, where you might stumble upon color palette inspiration. In fact, you might find it difficult to settle on a single palette, in which case it would make sense to save a selection of them and store them as arrays *within* another array. They would then become nested arrays, which are just arrays contained within other arrays. You might also see them referred to as 2D or multidimensional arrays.

Let's create a two-dimensional array containing three color palettes (illustrated in Figure 4-6). Beneath where we defined our initial grid group, include the following code (substituting in your own color palettes if you prefer):

```
// Create our colour palettes.
let palettes = [
  ['#5465FF', '#788BFF', '#9BB1FF', '#BFD7FF', '#E2FDFF'],
  ['#22577A', '#38A3A5', '#57CC99', '#80ED99', '#C7f9CC'],
  ['#4C5760', '#93A8AC', '#D7CEB2', '#A59E8C', '#66635B']
];
```

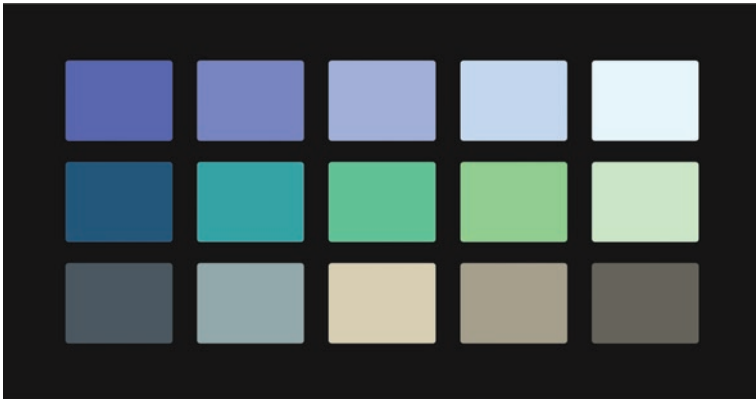


Figure 4-6. *Our three colour palettes*

Next, we're going to randomly select one of these palettes and store it in a variable that we can call on later.

```
// Pick a random palette.
let pickedPalette = Gen.random(palettes);
```

Now on to our grid variables. You can leave this as they were from the last sketch, but I'm not going to randomize the `gridSize` or spacing this time, as I want to place the emphasis on other randomized elements of the sketch.

```
let gridSize = 600;
let rows = Gen.random(3, 10);
let spacing = 10;
let increment = gridSize / rows;
let cellSize = Math.abs(increment - spacing);
```

Clipping Our Content

The next step in developing our grid-based composition will involve something called a `clipPath`. Essentially what a `clipPath` does is define the visible region of the shape it's applied to, in much the same way a scissors might define a cardboard cutout.

Instead of using a square to define our grid cells, we'll use a square-shaped `clipPath`. This way we can contain the content of our cells without them spilling out into adjacent cells. Let's step out our sketch for a moment to illustrate this. In Figure 4-7, we have a square-shaped `rect` that defines the `clipPath`, while the circle is the clipped element. On the right, you can see the effect of applying the `clipPath` to the circle.